2026.10.23 · KO

GovBiz

정부지원사업 탐색부터 신청 준비까지 연결하는 AI 서비스

SEARCH & RAG / ARCHITECTURE / DEPLOYMENT

검색

Elasticsearch + Qdrant + RRF

검증

AI 응답 검증 + 원문·인용 대조

구조

Spring Boot + FastAPI

배포

Vercel + AWS

목차

01 서비스 구조 03 - 09

02 애플리케이션 구조 10 - 13

03 검색·RAG와 근거 검증 14 - 20

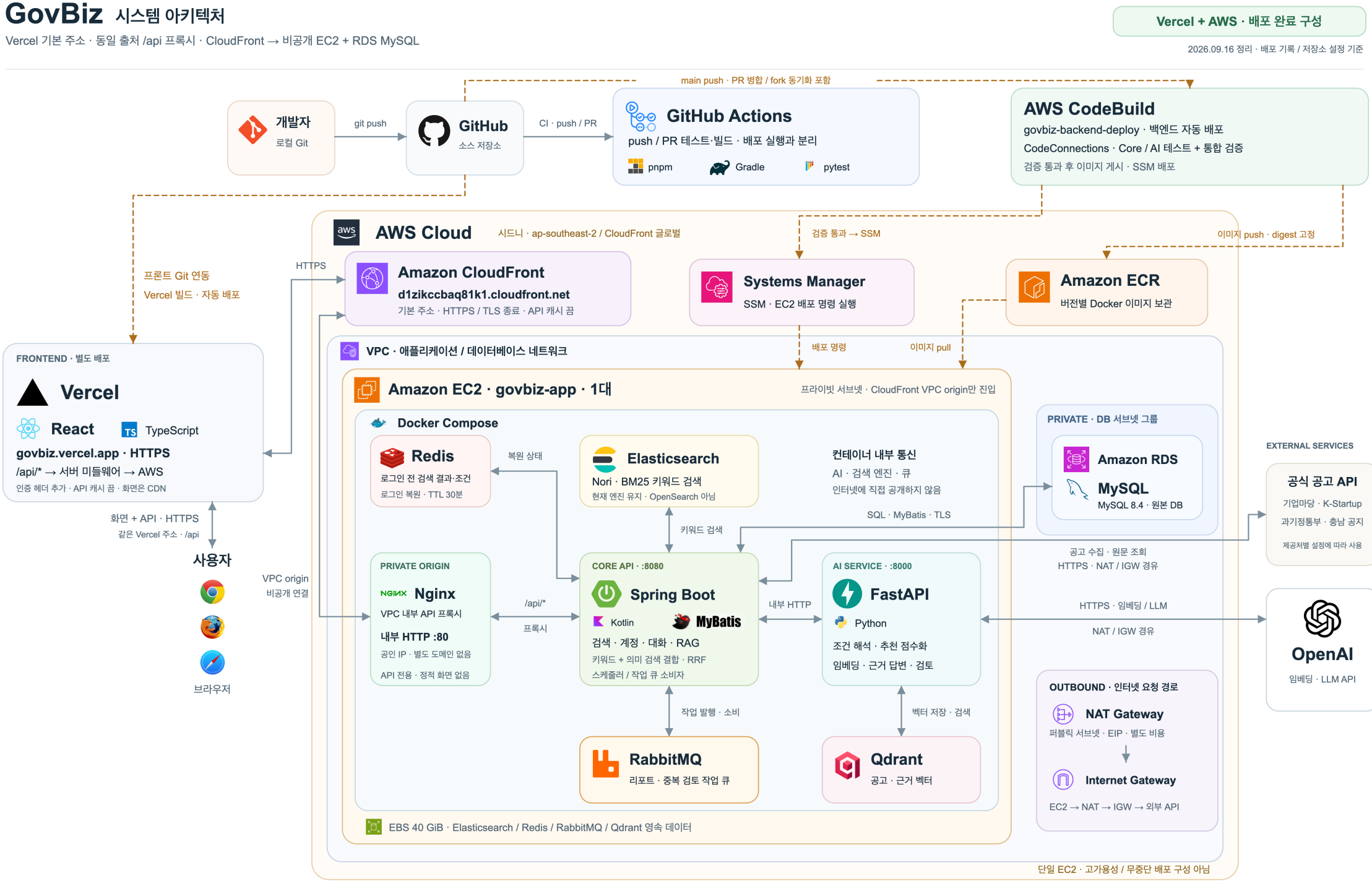
04 배포와 복구 21 - 23

공고 탐색을 신청 준비·진행 관리까지 연결합니다

여러 공식 제공처의 공고를 찾고 근거를 확인한 뒤, 관심함을 중심으로 문서 작성·일정·협업을 이어가는 서비스입니다.

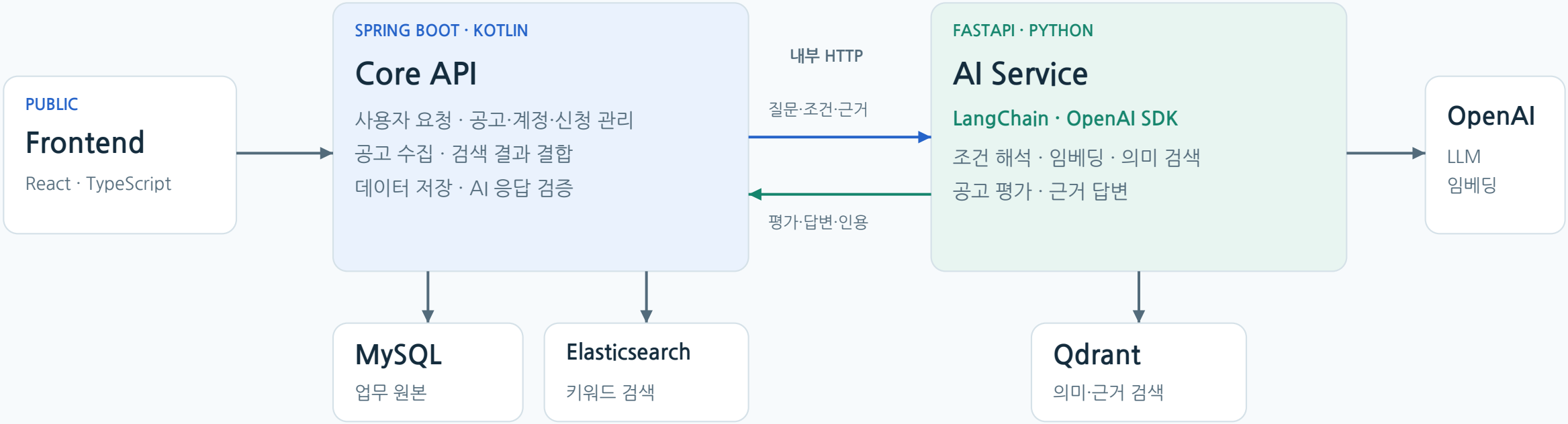


시스템 아키텍처


GovBiz 시스템 아키텍처
Vercel 기본 주소 · 동일 출처 /api 프록시 · CloudFront → 비공개 EC2 + RDS MySQL
Vercel + AWS · 배포 완료 구성
2026.09.16 정리 · 배포 기록 / 저장소 설정 기준
개발자
로컬 Git
git push
GitHub
소스 저장소
CI · push / PR
main push · PR 병합 / fork 동기화 포함
GitHub Actions
push / PR 테스트·빌드 · 배포 실행과 분리
pnpm
Gradle
pytest
AWS CodeBuild
govbiz-backend-deploy · 백엔드 자동 배포
CodeConnections · Core / AI 테스트 + 통합 검증
검증 통과 후 이미지 게시 · SSM 배포
프론트 Git 연동
Vercel 빌드 · 자동 배포
AWS Cloud
시드니 · ap-southeast-2 / CloudFront 글로벌
검증 통과 → SSM
이미지 push · digest 고정
HTTPS
Amazon CloudFront
d1zikccbaq81k1.cloudfront.net
기본 주소 · HTTPS / TLS 종료 · API 캐시 끔
Systems Manager
SSM · EC2 배포 명령 실행
Amazon ECR
버전별 Docker 이미지 보관
FRONTEND · 별도 배포
Vercel
React
TypeScript
govbiz.vercel.app · HTTPS
/api/* → 서버 미들웨어 → AWS
인증 헤더 추가 · API 캐시 끔 · 화면은 CDN
화면 + API · HTTPS
같은 Vercel 주소 · /api
사용자
브라우저
VPC origin
비공개 연결
VPC · 애플리케이션 / 데이터베이스 네트워크
배포 명령
이미지 pull
Amazon EC2 · govbiz-app · 1대
프라이빗 서브넷 · CloudFront VPC origin만 진입
Docker Compose
Redis
로그인 전 검색 결과·조건
로그인 복원 · TTL 30분
복원 상태
Elasticsearch
Nori · BM25 키워드 검색
현재 엔진 유지 · OpenSearch 아님
키워드 검색
컨테이너 내부 통신
AI · 검색 엔진 · 큐
인터넷에 직접 공개하지 않음
PRIVATE · DB 서브넷 그룹
Amazon RDS
MySQL
MySQL 8.4 · 원본 DB
SQL · MyBatis · TLS
EXTERNAL SERVICES
공식 공고 API
기업마당 · K-Startup
과기정통부 · 충남 공지
제공처별 설정에 따라 사용
공고 수집 · 원문 조회
HTTPS · NAT / IGW 경유
PRIVATE ORIGIN
Nginx
VPC 내부 API 프록시
내부 HTTP :80
공인 IP · 별도 도메인 없음
API 전용 · 정적 화면 없음
/api/*
프록시
CORE API · :8080
Spring Boot
Kotlin
MyBatis
검색 · 계정 · 대화 · RAG
키워드 + 의미 검색 결합 · RRF
스케줄러 / 작업 큐 소비자
내부 HTTP
AI SERVICE · :8000
FastAPI
Python
조건 해석 · 추천 점수화
임베딩 · 근거 답변 · 검토
HTTPS · 임베딩 / LLM
NAT / IGW 경유
OpenAI
임베딩 · LLM API
작업 발행 · 소비
벡터 저장 · 검색
RabbitMQ
리포트 · 중복 검토 작업 큐
Qdrant
공고 · 근거 벡터
OUTBOUND · 인터넷 요청 경로
NAT Gateway
퍼블릭 서브넷 · EIP · 별도 비용
Internet Gateway
EC2 → NAT → IGW → 외부 API
EBS 40 GiB · Elasticsearch / Redis / RabbitMQ / Qdrant 영속 데이터
단일 EC2 · 고가용성 / 무중단 배포 구성 아님
실선: 서비스 통신 · 주황 점선: 배포 경로 · CodeBuild는 Core / AI 이미지만 교체 · 운영 환경값·Compose는 별도 관리
기존 배포 확인 기록 + 저장소 설정 기준 · 작성 시 AWS 세션 만료로 실시간 상태 재조회 안 됨 · 단일 EC2 / 중단 가능 · OAuth·백업 복구·부하 검증 완료를 뜻하지 않음 · 비밀값 미표기

서비스 구조 — 화면·업무·AI 실행의 역할을 나눕니다

Python의 LangChain·OpenAI SDK를 활용하고, 업무 데이터 처리와 AI 실행의 책임을 분리했습니다.



React는 **화면**, Core API는 **업무 처리·검색 결합·응답 검증**, AI Service는 **의미 검색·LLM 실행**을 담당합니다.

데이터 구조 — 원본·검색 색인·임시 상태·작업 전달

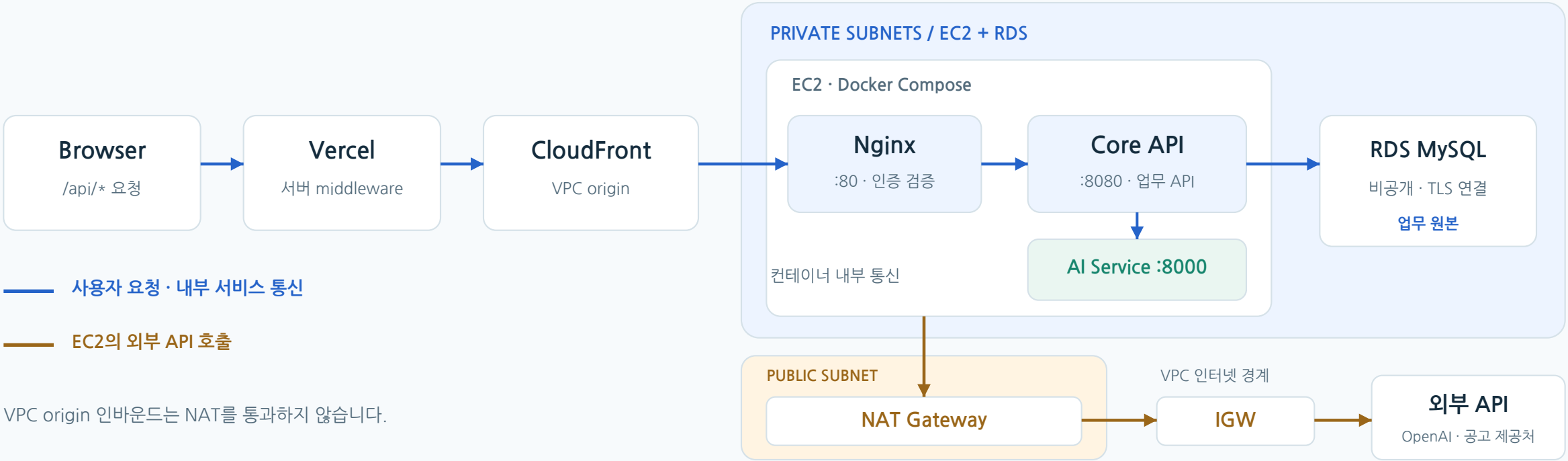
공고 원본과 업무 기록은 Core API가 관리합니다. AI Service는 MySQL에 직접 접근하지 않고 전달받은 데이터로 검색·생성을 실행합니다.



MySQL을 **원본**으로 두고, Elasticsearch·Qdrant는 **검색 색인**, Redis는 **임시 검색 결과 보관**, RabbitMQ는 **비동기 작업 전달**에 사용합니다.

운영 구조 — Vercel과 비공개 AWS를 연결합니다

EC2와 RDS는 프라이빗 서브넷에 둡니다. 사용자 API 요청은 Vercel과 CloudFront를 거치고, 서버의 외부 호출은 NAT Gateway를 사용합니다.



백엔드 배포 CodeBuild 검증·빌드 → ECR 이미지 저장 → SSM 배포 명령 → EC2 서비스 교체

프론트엔드는 **Vercel**에서 제공하고, EC2·RDS는 **프라이빗 네트워크**에 배치했습니다. 백엔드는 **CodeBuild·ECR·SSM**을 통해 배포합니다.

검색 준비를 마친 최신 수집 결과만 반영합니다

검색 데이터를 모두 준비한 뒤, 같은 제공처에서 가장 최근에 시작한 작업인지 확인합니다.



검색 결과 확인 현재 MySQL에서 제공하는 공고와 내용이 같은 검색 결과만 사용합니다.

검색 데이터 준비 순서

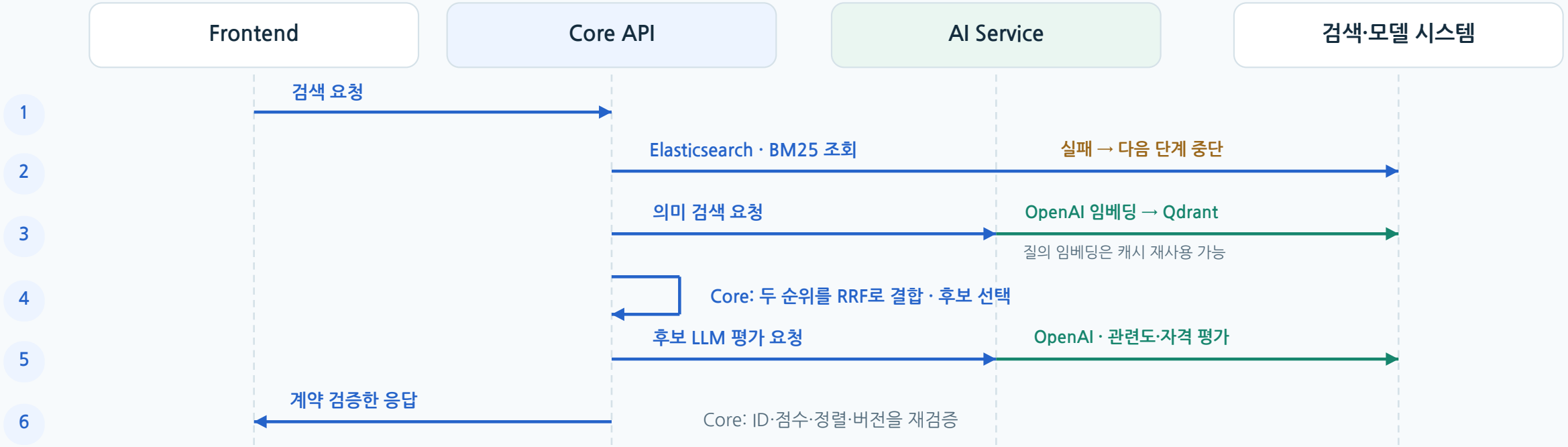
Elasticsearch 준비 후 Qdrant를 준비합니다.
40건이면 16·16·8건으로 요청하며, 공고마다 벡터를 따로 준비합니다.

작업 번호 비교 예시

1번 작업 뒤에 2번이 시작됐다면 1번 결과는 반영하지 않습니다.
2번이 실패해도 마지막으로 정상 반영한 공고를 유지합니다.

검색 호출 순서로 비용과 장애 경계를 통제합니다

Core가 검색 순서를 조율합니다. Elasticsearch 성공 뒤에 의미 검색을 호출하고, 후보를 합친 뒤 LLM으로 평가합니다.



요청 접수·외부 통신·DB 접근의 역할을 나눴습니다

지원사업 기능(supportprogram) 안에서 역할을 나누고, Service가 업무에 필요한 경로를 선택합니다.

Controller
HTTP 요청을 받고 응답

→

Service
조회·호출 순서와 업무 흐름 조율

필요한 업무에 따라 아래 경로를 사용
외부 정보 요청 또는 저장된 데이터 조회·저장

외부 정보가 필요할 때

단순 호출은 Facade 없이 Client로

Service
외부 정보 요청

→

Facade · 필요한 경우만
복잡한 응답 검증·변환

→

Client
외부 서버와 HTTP 통신

→

외부 API
AI Service·기업마당

저장된 공고를 읽거나 쓸 때

Service
조회·저장 요청

→

Repository
조회·저장, 데이터 변환

→

MyBatis Mapper
SQL 실행 요청

→

XML
실행할 SQL

→

MySQL
공고 데이터

Repository 경계: 업무 모델 ↔ DbRow · DB 행 형식은 외부로 내보내지 않음

Domain · 프레임워크와 무관한 업무 규칙
예: 신청 기간과 오늘 날짜로 접수 상태 계산

예: 공고 목록 보기 → DB 조회 경로 사용
저장된 공고 조회 → 접수 상태 계산 → 필터·정렬 → 목록 응답

화면·업무·서버 통신의 역할을 나눴습니다

주요 업무 기능의 설계 기준과 대표 조회 흐름입니다. 모든 기능에 일관되게 적용하지는 못했습니다.

01 역할과 코드 의존 방향

Presentation · 화면

React View + ViewModel
사용자 동작 · 입력 확인 · 결과 표시
입력 확인: 필수값·글자 수 등

Domain · 업무

UseCase: 업무 흐름 처리
Repository 계약: 필요한 조회 기능의 약속
구체적인 HTTP 구현을 직접 알지 않음

Data · 서버 통신

Repository 구현체 · HTTP 호출
Zod로 서버 응답의 형식 검증
DTO → 업무 모델 변환

코드 의존 **Presentation** → **Domain** ← **Data** Data가 Domain의 계약을 구현

02 실제 조회 흐름

예: 사용자가 지역 필터를 변경합니다.

ViewModel
조회 요청

→

UseCase
조건 정리·검사

→

Repository 구현체
계약에 맞춰 HTTP 호출

→

Core API
조회 결과 응답

응답 처리 Data: Zod 검증 → DTO를 업무 모델로 변환 → UseCase에 반환 → ViewModel이 화면 상태 갱신

03 Awilix 연결

Data의 Repository 구현체 → Domain의 UseCase에 주입

앱 구성 단계에서 연결

MVVM — ViewModel로 화면과 업무 호출 분리

공고 목록 화면의 예: 화면은 표시하고, ViewModel은 조회와 상태를 연결합니다.

Presentation

View · 화면

목록 · 로딩 · 오류 표시

필터 UI · 이벤트 연결

조건·재시도 →

← 화면 상태

Presentation

ViewModel

useEffect로 UseCase 실행

조건 변경 · 재시도 시 조회

useState로 조회 상태 관리

결과 · 로딩 · 오류 · 재시도 제공

업무 호출 →

← 조회 결과

Domain

Model · 업무

BrowseSupportProgramsUseCase

입력 정리 · 검사

Repository 계약으로 데이터 조회

ViewModel이 화면에 제공

조회 상태(phase) · 결과(data)

필터 옵션 · 재시도(retry)

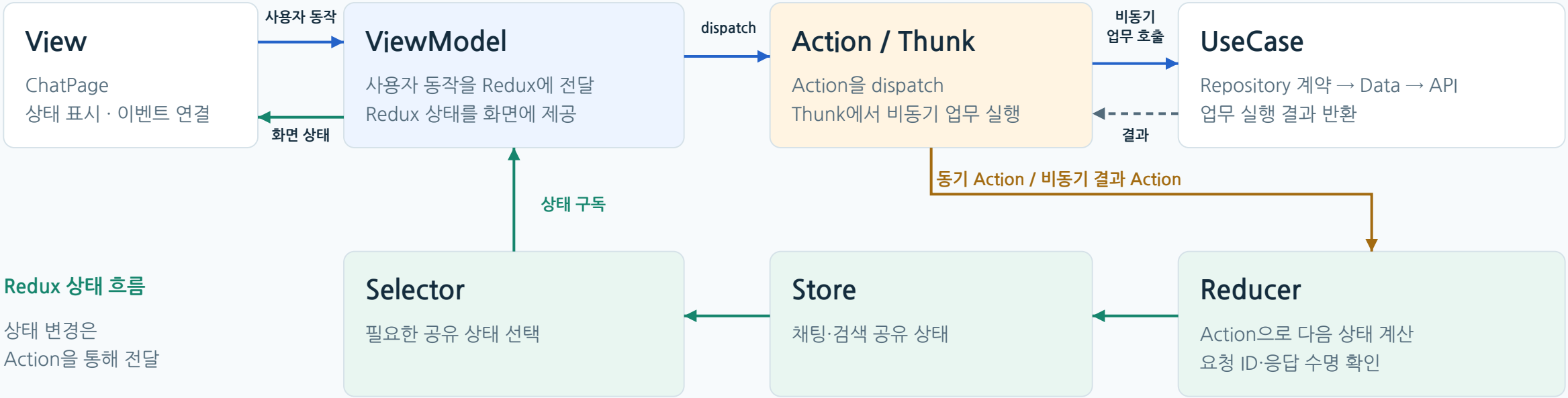
역할의 경계

View는 UseCase·Repository를 직접 실행하지 않습니다.

업무 규칙은 UseCase에 둡니다.

Flux: Redux의 상태 흐름도 ViewModel을 통해 연결합니다

채팅 화면의 예: ViewModel을 거쳐 공유 상태를 한 방향으로 갱신합니다.



Redux 상태 흐름

상태 변경은
Action을 통해 전달

ViewModel로 비즈니스로직 상태관리 분리

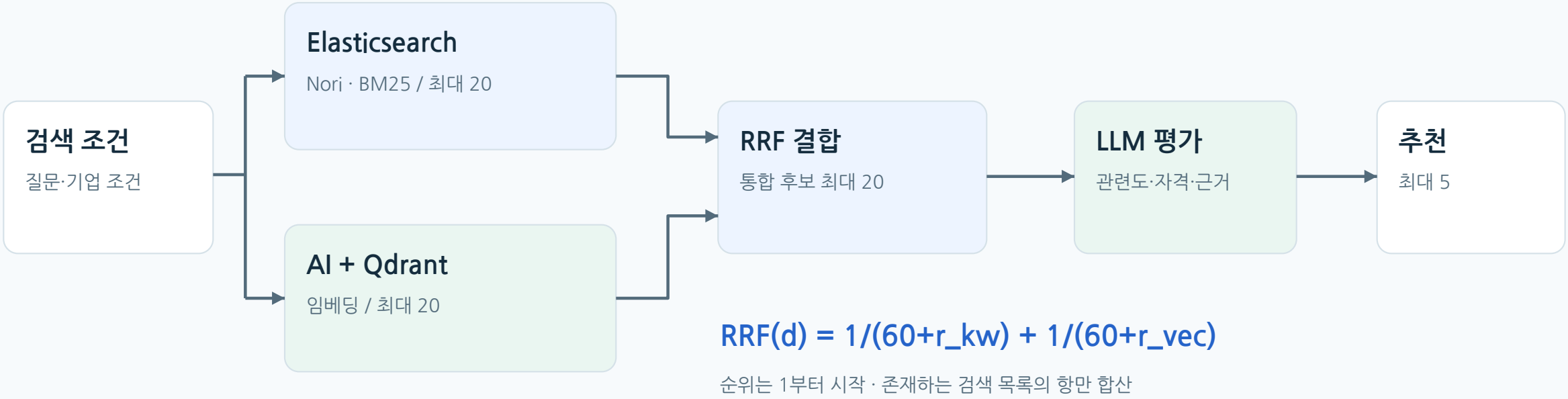
View는 ViewModel을 통해 동작을 요청하고 상태를 받습니다.
메시지·검색 등 공유 상태는 Redux에서 관리합니다.

Reducer는 상태 계산만 담당

외부 API 호출은 Thunk가 UseCase에 위임합니다.
Reducer 안에서는 외부 API를 호출하지 않습니다.

키워드와 의미 후보를 RRF 순위로 결합합니다

서로 다른 검색 점수의 척도를 직접 더하지 않고, 두 목록의 순위를 동일 가중치로 통합합니다.



$$RRF(d) = 1/(60+r_{kw}) + 1/(60+r_{vec})$$

순위는 1부터 시작 · 존재하는 검색 목록의 항만 합산

실제 호출은 Elasticsearch → AI 의미 검색 순서입니다. 키워드 검색 장애를 유료 질의 임베딩 전에 확인합니다.

공고를 찾는 벡터와 답변 근거 벡터를 분리합니다

Qdrant를 공유해도 검색 목적과 단위가 다릅니다. 공고 검색과 공식 원문 근거 검색은 서로 다른 컬렉션에서 실행합니다.

DISCOVERY
공고 단위 검색
공식 API 4개 제공처
관련 공고를 찾는 단계

→

PREPARE
Core 검색 문서
제목·기관·대상·지역
신청 기간·지원 내용

→

EMBED
AI 임베딩
검색 문서 1건마다
벡터 1개

→

QDRANT
공고 컬렉션
공고 ID + 내용 해시
제공처 코드 + 벡터

EVIDENCE
원문 근거 검색
기업마당 상세 HTML
답변의 근거를 찾는 단계

→

PREPARE
Core 정제·청킹
최대 1,500자 / 청크
문서당 최대 50개

→

EMBED
AI 임베딩
원문 청크 1개마다
벡터 1개

→

QDRANT
근거 컬렉션
문서·청크 ID / 해시·순서
청크 원문 + 벡터

같은 문서 버전·임베딩 설정의 벡터는 재사용하고 누락분만 생성합니다. 질문 벡터는 검색에만 사용합니다.

두 검색에 함께 오른 후보가 어떻게 합쳐지는지 봅니다

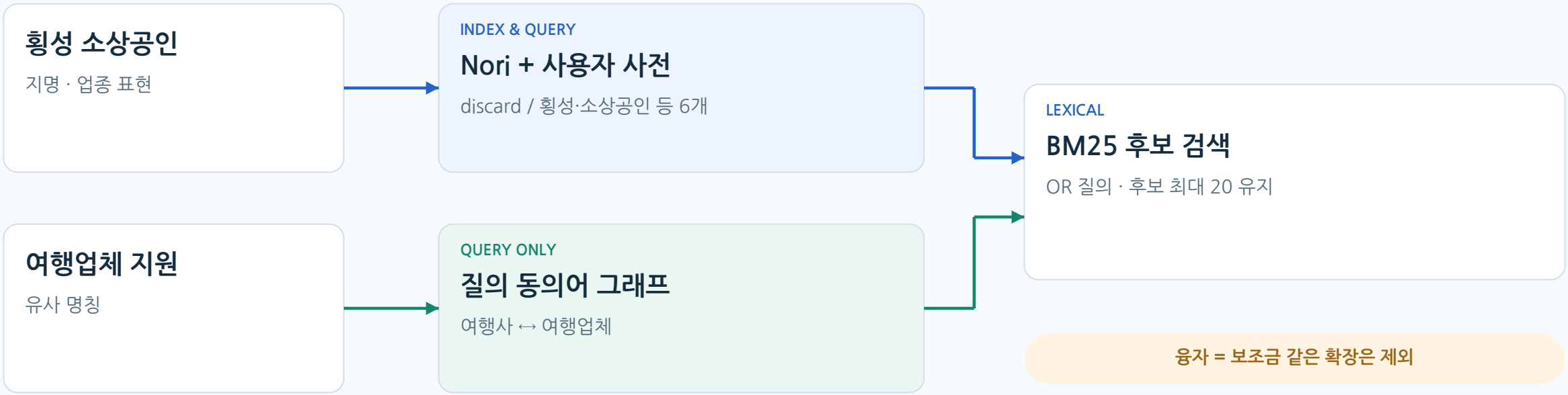
설명용 순위를 직접 계산한 예시입니다. 실제 공고의 검색 결과나 성능 측정치는 아닙니다.



형태소 분석과 유사 표현의 누락을 단계별로 좁힙니다

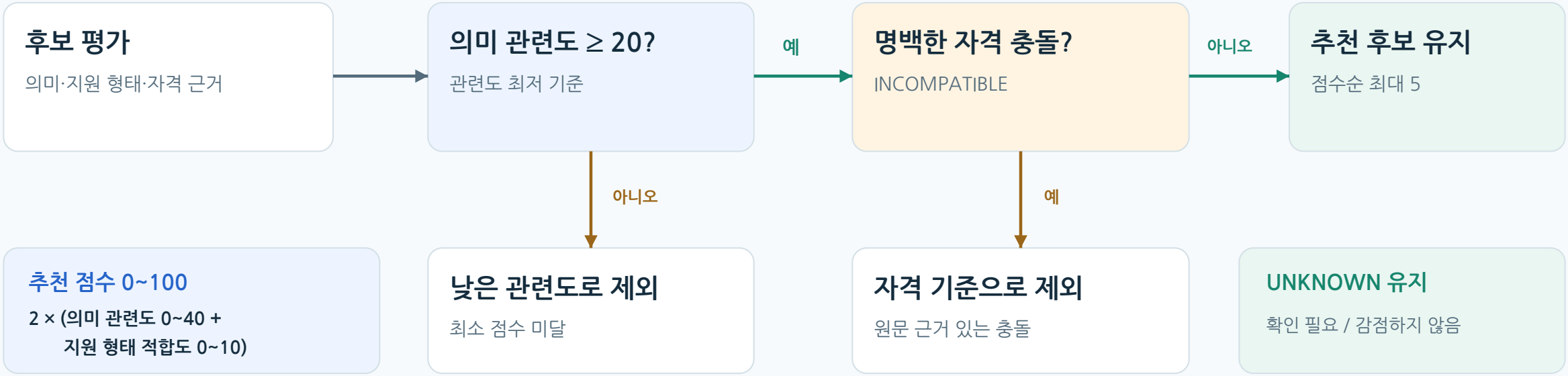
분해 모드·사용자 사전·질의 동의어를 보완하고, 금융 방식이나 신청 범위를 바꾸는 확장은 제외했습니다.

입력의 표현 차이



관련도 기준과 자격 기준을 서로 다른 분기로 적용합니다

자격을 확인하지 못한 후보는 관련도에서 감점하지 않습니다. 그림은 선별 규칙의 요약이며 호출 순서도는 아닙니다.



추천 점수 100점은 선정 확률이 아닙니다. 자격 정보가 부족하면 UNKNOWN으로 유지해 추가 확인이 필요함을 표시합니다.

선택한 공고의 원문에서 답변 근거를 찾습니다

Core와 AI Service가 원문을 준비하고, 질문에 필요한 구간을 확인해 답변합니다.

예시 질문 “신청 기간이 언제인가요?” → 신청 기간이 적힌 원문 구간을 찾아 답변에 활용

01-03 원문과 검색 데이터를 준비합니다

01 CORE

공식 원문 확보

선택한 기업마당 공고의
원문을 가져오거나 재사용

→

02 CORE

원문을 구간으로 나누기

구간당 최대 1,500자,
공고당 최대 50개 구간

→

03 AI SERVICE

구간별 벡터 준비

구간을 벡터로 만들어 Qdrant에 저장
이미 같은 벡터가 있으면 재사용

04-06 질문에 맞는 근거를 찾고 답변합니다

04 CORE → AI SERVICE

질문과 검색 범위 전달

질문과 현재 공고의 구간 목록 전달
Qdrant에서 관련 구간 최대 5개 검색

→

05 CORE

검색 결과 확인

돌아온 구간 ID·해시·순서를 대조
Core가 가진 원문 구간을 복원

→

06 AI SERVICE → CORE

답변과 출처 확인

AI가 검증된 원문으로 답변 작성
Core가 인용을 확인해 출처 제공

인용한 구간이 실제 전달된 원문에 속하는지 확인합니다. 답변 전체의 의미적 정확성까지 보장하지는 않습니다.

검색 결과와 답변의 인용을 두 번 확인합니다

Core가 검색 직후 원문 구간을 확인하고, 답변 생성 후 인용 대상을 다시 확인합니다.

예시 “신청 기간이 언제인가요?” → 검색 결과에 신청 기간이 담긴 구간 A가 있다고 가정

01 검색 직후

찾은 구간이 현재 공고의 원문인가?

AI가 구간 A의 ID·내용 해시·문서 ID·순서를 반환

Core가 현재 공고의 구간 목록과 각각 대조

중복·결과 수·점수·정렬 상태도 확인

통과하면 Core가 가진 구간 A의 원문을 복원

목록과 다르거나 검증에 실패하면 오류 처리

02 답변 생성 후

답변이 실제 전달한 구간을 인용했나?

AI가 검증된 원문으로 답변하고 인용 ID를 반환

Core가 인용 ID가 전달한 구간에 있는지 확인

인용 중복과 답변 상태에 맞는 인용 여부도 확인

통과하면 Core의 원문 발췌와 공식 출처를 제공

근거 부족은 인용 없이 표시 · 검증 실패는 오류

AI는 전달받은 목록의 번호를 선택하고 AI Service가 실제 구간 ID로 변환합니다.

인용의 출처·형식을 확인하는 절차이며, 답변 전체의 의미적 정확성을 보장하지는 않습니다.

코드를 검사하고 이미지를 EC2에 배포합니다

프론트엔드는 Vercel에서 배포하고, 백엔드는 CodeBuild가 아래 과정을 실행합니다.

시작 main에 코드 반영 → CodeBuild 실행 · CodeConnections로 GitHub 소스 연결

01
검증 게이트

Core 빌드·테스트
AI 테스트·패키지 빌드
Compose·Nginx 등 통합 확인
실패하면 배포 중단

02
이미지 준비

Core·AI 실행 이미지 준비
커밋 ID를 이미지 태그로 지정
예: git-abc123…
같은 태그가 이미 있으면 재사용

03
ECR 게시

이미지를 ECR에 저장
저장된 이미지의 digest 확인
digest로 이미지 주소를 고정

04
SSM 배포

CodeBuild가 SSM에 명령 요청
지정 EC2에 이미지 주소 전달
EC2의 배포 스크립트 실행
서비스 교체·상태 확인 → 22쪽

커밋 태그와 digest

커밋 태그: 어떤 코드에서 만든 이미지인지 표시

digest: 실제 이미지 내용을 식별하는 고유값

오래된 배포 방지

ECR 게시 전과 후 최신 main 커밋인지 확인

이미 오래된 빌드는 EC2에 배포하지 않음

통합 검증의 OpenAI 호출은 스텁을 사용합니다. 테스트 통과가 실제 LLM 답변 품질을 보장하지는 않습니다.

배포 전 확인부터 상태 검증·복구까지

EC2 배포 스크립트가 확인과 교체를 순서대로 실행합니다.

01 사전 점검

배포 전에 확인

다른 배포가 동시에 실행되지 않도록 잠금
Docker 저장 공간 4 GiB 이상 확인
새 이미지의 저장소·커밋·실행 환경 확인
기존 서비스와 이미지 설정 상태 확인

02 서비스 교체

AI → Core 순서

AI를 교체하고 정상 실행을 확인
Core를 교체해 정상 실행을 확인
Nginx 설정을 검사하고 다시 불러옴

03 상태 검증

교체 후 다시 확인

Core·AI·Nginx 등 서비스 상태 확인
Core에서 AI 상태 확인 API를 호출
AI 연결 결과가 정상(up)인지 확인
Qdrant는 실행 중인지 확인

교체·상태 확인 중 실패하면

배포 전 환경 파일을 복원하고 이전 이미지로 재실행 시도
복구에 성공해도 이번 배포는 실패로 기록

유지하는 대상

검색 엔진·캐시·큐 컨테이너와 영속 데이터 유지
운영 환경·Compose 전체를 자동으로 덮어쓰지 않음

Nginx 설정·상태 확인은 외부 사용자 요청의 성공까지 보장하지 않습니다.
서버 한 대를 교체하는 동안 서비스가 중단될 수 있으며, 이미지 복구로 DB 변경을 되돌리지는 않습니다.

교체 실패와 복구 실패를 별도 결과로 남깁니다

배포는 상태 확인을 거쳐 종료합니다. 이전 이미지 복구에 성공해도 이번 배포가 성공한 것으로 바꾸지 않습니다.

